베이스라인 아키텍처

생성일	2026년 5월 8일 오후 3:30
태그	

📂 1. 프로젝트 파일 및 폴더 구조 (모듈화)

- `main.cpp` : [사령탑] 전체 흐름(Phase 1 ~ 5)을 지휘하고 시간을 재는 메인 파일.
- `models.hpp` : [데이터 그릇] `주문(Order)`, `경로(Bundle)`, `전체 정답(Solution)` 같은 **핵심 데이터** 구조체들만 모아두는 곳.
- `phase1_promising.hpp` : [내비게이션] 주문 간의 **유망 점수(Matrix)**를 계산하고 저장하는 로직.
- `phase2_init.hpp` : [**초기화**] 텅 빈 상태에서 어떻게든 첫 번째 배달 시간표를 억지로 끼워 맞추는 로직.
- `phase3_alns.hpp` : [**ALNS** 엔진] 잘 짜인 경로를 부수고(Destroy), 점수판을 보고 더 좋은 곳에 다시 끼워 넣는(Repair) 로직.
- `phase4_rider.hpp` : [가성비 챙기기] 완성된 경로의 **배달원**을 도보/오토바이/차량 중 가장 싼 사람으로 쓱 바꿔치기하는 로직.
- `utils.hpp` : [도구상자] 엑셀/CSV 데이터 읽어오기, 거리 계산기 등 잡다한 함수들.

📦 2. 핵심 데이터 구조

1. `Order` (주문 정보)

원본 데이터에는 위도, 경도, 주문 시각, 준비 시간 등 정보가 9개나 있지만,
알고리즘이 실제로 쳐다볼 정보는 **3개**뿐이다. (거리는 `DIST` 행렬이 다 알려줌!)

- **설계 포인트:** 데이터를 읽어올 때 미리 `주문발생시각 + 주문 준비 시간 = readytime`을 계산해서 저장해 둬. 알고리즘 돌 때마다 덧셈할 시간을 아끼는 거지.
- **변수 구성:**
  - `id`: 주문 번호 (0 ~ K-1)
  - `readytime`: 픽업 가능 시각 (미리 더해둔 값)
  - `volume`: 음식 부피 (용량 제약 체크용)
  - `deadline`: 배달한계시각

2. `Rider_Info` (배달원 스펙)

도보, 오토바이, 차량의 스펙을 매번 하드코딩할 수 없으니,
데이터를 읽어와서 저장해 두는 Dictionary 같은 구조체

- **변수 구성:**
  - `type`: "WALK", "BIKE", "CAR"
  - `speed`: 속도 (거리 계산 후 시간으로 나눌 때 사용)
  - `capacity`: 최대 적재 용량 (100 등)
  - `var_cost`: 100m당 변동비
  - `fixed_cost`: 고정비
  - `service_time`: 접근 시간 (120초 등)
  - `availability`: 가용 수 (이건 루프 돌 때 차량 무한 리필(?)을 위해 참고)

3. `Bundle` (배달원 1명의 스케줄)

ALNS 루프(Phase 3)에서 미친 듯이 쪼개고 합칠 번들

- **설계 포인트:** 경로(`route`)는 0~2K-1 사이의 숫자 배열로 저장함.

  예) 주문 1과 2를 묶는다면 `route = [1, 2, 1+K, 2+K]`
  (1번 픽업 -> 2번 픽업 -> 1번 배달 -> 2번 배달) 형태로 노드 번호를 저장

- **변수 구성:**
  - `rider_type`: 현재 배정된 배달원 ("BIKE" 등)
  - `route`: 방문 순서 리스트 (픽업 노드와 배달 노드가 섞인 배열)
  - `cost`: 이 경로를 수행하는 데 드는 총비용 (고정비 + 변동비)
  - `total_volume`: 현재 가방에 담긴 총부피 (용량 제약 튕겨내기용)

4. `Solution` (전체 배달 시간표)

Phase 5에서 솔버(Gurobi)에게 "이게 지금 제일 싼 조합이야!" 하고 던져줄 최종 정답지

- **변수 구성:**
  - `bundles`: 만들어진 `Bundle`들의 리스트
  - `total_cost`: 모든 `Bundle`의 `cost`를 다 합친 총비용 (목적 함수 값!)

💡 요약

1. 프로그램이 켜지면 데이터를 쭉 읽어온다.
2. 위도, 경도 같은 쓸데없는 건 다 버리고, `Order` **구조체에 알맹이(시간, 부피)만 쏙쏙 뽑아 담는다.**
3. 거리 행렬은 전역 변수로 `DIST[2K][2K]`에 얌전히 모셔둔다.
4. 이제 `Bundle` 구조체 안에 `Order`들을 넣었다 뺐다 하면서 알고리즘을 돌린다!

⚙️ 3. 메인 흐름도 (main.cpp의 시나리오)

1. **[준비 태세] 데이터 로드 및 초기 세팅**
   - `utils.hpp`를 써서 문제 데이터(주문 내역, 거리 행렬)를 싹 읽어옴.
2. **Phase 1: 점수판 생성 (단 1회 실행)**
   - 모든 주문끼리의 궁합 점수를 계산해서 2차원 표(Matrix)로 메모리에 박아둠.

3. **Phase 2: 초기 해 생성 (단 1회 실행)**
    - 점수판을 힐끗힐끗 보면서, K개의 주문을 모조리 `Bundle`들에 쑤셔 넣어서 첫 번째 `Solution`을 뚝딱 만듦.
4. **Phase 3 & 4: 맹렬한 탐색 루프 (제한 시간 동안 무한 반복)**
    - *타이머 시작! (예: 10초)*
    - 현재 `Solution` 복사.
    - **Phase 3 [ALNS]:** 몇 개의 주문을 뽑아내고(**Destroy**), 점수판을 보고 싼 곳에 다시 넣음(**Repair**).
    - **Phase 4:** 새로 짜인 경로들의 배달원을 가장 싼 타입으로 교체.
    - *평가:* 방금 수정한 `Solution`이 예전보다 싸면(혹은 확률적으로) Accept! 아니면 버림.
    - 이 과정에서 만들어진 예쁜 `Bundle`들은 'Gurobi 창고'에 차곡차곡 복사해 둠.
5. **Phase 5 [SP]: 마무리 및 솔버로 토스**
    - 루프가 끝나면, 'Gurobi 창고'에 쌓인 수만 개의 `Bundle`들을 파이썬이 읽을 수 있게 CSV나 JSON으로 포장해서 출력(Export)하고 C++프로그램 종료.

C:\Users\hcy05\AppData\Local\Programs\Python\Python313\python.exe run_pipeline.py